

PORTADA

# Arquitectura de Software

## Semana 5 — Stack Tecnológico

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

**13 empleados**  
**30 millones de usuarios**



# Instagram, abril de 2012

|                          |                                                                         |
|--------------------------|-------------------------------------------------------------------------|
| Empresa                  | Instagram — red social de fotografía móvil                              |
| Adquisición por Facebook | Abril de 2012, por mil millones de dólares                              |
| Tamaño del equipo        | 13 empleados en el momento de la adquisición                            |
| Usuarios                 | Más de 30 millones de usuarios registrados                              |
| Backend                  | Python con el framework <b>Django</b>                                   |
| Base de datos            | PostgreSQL, con particionamiento (sharding) horizontal entre servidores |
| Infraestructura          | Amazon Web Services (EC2), sin servidores propios                       |

Caso documentado en el blog público de ingeniería de Instagram y en cobertura de prensa tecnológica de la adquisición.

CASO · LA DECISIÓN DE STACK

## Un conjunto de tecnologías, no piezas sueltas

Instagram no eligió cada tecnología de forma aislada: eligió un **conjunto coherente** —lenguaje, framework, base de datos e infraestructura— pensado para trabajar bien en conjunto. Ese conjunto integrado es lo que llamamos **stack tecnológico**.

- **Django** (framework Python, visto su equivalente Rails en la Semana 2) les dio productividad: pocas personas, mucho alcance.
- **PostgreSQL** les dio garantías fuertes de consistencia de datos, clave para una red social con relaciones entre usuarios.
- **AWS** les permitió escalar infraestructura sin comprar ni administrar servidores propios.

CASO · POR QUÉ FUNCIONÓ CON UN EQUIPO TAN PEQUEÑO

## El stack como multiplicador de productividad

Con solo 13 personas atendiendo 30 millones de usuarios, cada decisión de stack tenía que **ahorrar trabajo**, no generarlo. Django ofrecía convenciones ya resueltas (como Rails); PostgreSQL ya incluía particionamiento y replicación maduros; AWS eliminaba la necesidad de administrar centros de datos propios.

**El stack correcto no es el más popular — es el que necesita menos esfuerzo del equipo disponible para sostener el crecimiento real del producto.**

TRANSICIÓN

## Instagram eligió Python/Django + PostgreSQL. Otros equipos, otras combinaciones.

Con el tiempo, ciertas combinaciones de tecnologías se repitieron tanto entre proyectos que recibieron **nombres propios**, fáciles de comunicar en una entrevista de trabajo o en una propuesta técnica.

**Hoy conocemos los stacks con nombre propio más comunes, y los criterios reales para elegir uno — cerrando así la Unidad 1 del curso.**

## CONCEPTO

# ¿Qué es un stack tecnológico?

Un **stack tecnológico** es el conjunto de tecnologías —sistema operativo o infraestructura, servidor web, base de datos, lenguaje y framework de backend, y a veces framework de frontend— que, juntas, permiten construir y ejecutar una aplicación completa.

Es una decisión arquitectónica (Semana 3): afecta a todo el sistema y es costosa de cambiar una vez el proyecto avanza.

## CONCEPTO

# LAMP: el stack pionero de la web

**LAMP** (Linux, Apache, MySQL, PHP/Perl/Python) es de los stacks con nombre propio más antiguos, popular desde finales de los 90. Sistemas masivos como los primeros años de **Facebook** y **Wikipedia** se construyeron sobre variantes de LAMP.

- **Linux** — sistema operativo del servidor
- **Apache** — servidor web
- **MySQL** — base de datos relacional
- **PHP** (o Perl, o Python) — lenguaje de backend



CONCEPTO

Variantes: WAMP y XAMPP

| Stack | Diferencia con LAMP                                                                                                      |
|-------|--------------------------------------------------------------------------------------------------------------------------|
| WAMP  | Igual, pero sobre <b>Windows</b> en vez de Linux — común en entornos de desarrollo local                                 |
| XAMPP | Multiplataforma (Windows, macOS, Linux); agrega herramientas de instalación simplificada y soporte para Perl junto a PHP |

Ambas variantes se usan sobre todo para desarrollo y pruebas locales, no necesariamente en producción.

## CONCEPTO

# MEAN y MERN: el stack todo-JavaScript

Con la llegada de Node.js (Semana 2), surgió la posibilidad de usar **un solo lenguaje —JavaScript— en todo el stack**, del navegador al servidor.

| Stack | Componentes                                                                                                  |
|-------|--------------------------------------------------------------------------------------------------------------|
| MEAN  | MongoDB (base de datos NoSQL) + Express (framework backend) + Angular (frontend) + Node.js (runtime backend) |
| MERN  | Igual, pero con React en vez de Angular para el frontend                                                     |

La ventaja declarada de estos stacks es reducir el cambio de contexto: el mismo lenguaje en frontend y backend.

CONCEPTO

# Comparados por capa

| Capa              | LAMP                        | MEAN / MERN          | Instagram (2012)               |
|-------------------|-----------------------------|----------------------|--------------------------------|
| Lenguaje backend  | PHP                         | JavaScript (Node.js) | Python                         |
| Framework backend | Variable (Laravel, Symfony) | Express              | Django                         |
| Base de datos     | MySQL (relacional)          | MongoDB (NoSQL)      | PostgreSQL (relacional)        |
| Frontend          | HTML/JS tradicional         | Angular / React      | Nativo móvil + web tradicional |

CONTRAEJEMPLO

## Mito: "el mejor stack es el más popular"

Es tentador elegir el stack de moda —hoy, con frecuencia, algún sabor de MERN— asumiendo que si lo usan las empresas grandes, es la opción correcta para cualquier proyecto.

**Instagram no usó MEAN ni el stack más hablado de su momento.** Eligió Python/Django + PostgreSQL porque encajaba con las habilidades de su equipo pequeño y con las garantías de consistencia que necesitaba una red social. **El stack correcto depende del contexto, no de la popularidad.**

## CONCEPTO

# Criterios reales para elegir un stack

- **Habilidades del equipo:** el stack más "correcto" en teoría no sirve si nadie en el equipo lo domina, ni hay tiempo para aprenderlo.
- **Naturaleza de los datos:** relaciones estrictas entre entidades favorecen bases de datos relacionales (como PostgreSQL en Instagram); datos muy variables favorecen NoSQL.
- **Requisitos no funcionales** (Semana 4): escalabilidad esperada, latencia tolerable, presupuesto de infraestructura.
- **Madurez y ecosistema:** documentación, comunidad, disponibilidad de librerías para el dominio del proyecto.



CONCEPTO · APLICADO AL CASO

# Por qué Instagram no usó un stack de nombre propio

|                         |                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------|
| Habilidades del equipo  | Los fundadores ya dominaban Python de proyectos previos                                         |
| Naturaleza de los datos | Relaciones entre usuarios, fotos y "me gusta" — encajaban mejor con un modelo relacional        |
| NFR                     | Necesitaban escalar muy rápido con muy poco personal — Django y AWS reducían la carga operativa |

Ninguno de los stacks con nombre propio (LAMP, MEAN) encajaba exactamente — así que combinaron piezas maduras según sus criterios reales.

## TENSIÓN CONCEPTUAL

### ¿Stack estándar completo, o combinación "a la carta"?

**Postura 1:** seguir un stack con nombre propio completo (ej. MERN) da coherencia, documentación abundante y facilidad para contratar personas que ya lo conocen.

**Postura 2:** como Instagram, combinar piezas de distintos "stacks con nombre" según el criterio de cada capa puede ajustarse mejor al problema real, a costa de menos referencias listas para copiar.

¿Qué deberían priorizar en el Sprint 1 de este curso: un stack estándar reconocible, o piezas elegidas una por una según necesidad?



## INTERACCIÓN

# Elijan un stack, con criterio

**Para cada escenario, propongan un stack (con nombre propio o combinado) y justifiquen con al menos dos de los criterios de la diapositiva anterior:**

1. Un equipo de 2 personas construyendo el MVP de una app de reservas en 6 semanas.
2. Un sistema bancario interno con reglas de consistencia muy estrictas entre cuentas.
3. Una red social nueva con datos muy variables entre usuarios (perfiles con campos distintos entre sí).

5 minutos · respondan en voz alta o en el chat



## SÍNTESIS

# Ideas clave de hoy — y cierre de la Unidad 1

1. Un **stack tecnológico** es un conjunto coherente de tecnologías por capa — sistema operativo/infraestructura, servidor, base de datos, backend y frontend.
2. **LAMP, WAMP, XAMPP, MEAN y MERN** son combinaciones con nombre propio, cada una con supuestos distintos sobre el tipo de proyecto.
3. Instagram (2012) demuestra que el stack correcto depende de las habilidades del equipo, la naturaleza de los datos y los requisitos no funcionales — no de la popularidad.
4. Elegir un stack es una **decisión arquitectónica** (Semana 3): afecta a todo el sistema y es costosa de revertir.
5. Con esto cerramos la **Unidad 1**: de "dónde vive el código" (Semana 1) a "con qué tecnologías concretas se construye" (hoy).

## INSTRUCCIONES

# Taller de hoy — Implementación de ejemplo

### **Demostración guiada por el docente — 30 a 40 minutos**

El docente ubica, en vivo, el stack usado en los ejemplos de código de todas las sesiones anteriores (Spring Boot + HTML/JS) dentro de la tabla comparativa de la diapositiva 11, y muestra qué pieza faltaría agregar para completar un stack con nombre propio equivalente.

INSTRUCCIONES · EJEMPLO COMENTADO

# El stack de este curso, capa por capa

| Capa          | Lo usado en clase                          | Rol en el stack                                                   |
|---------------|--------------------------------------------|-------------------------------------------------------------------|
| Backend       | Java + Spring Boot                         | Equivalente al "P" de LAMP o al Express de MEAN, en otro lenguaje |
| Frontend      | HTML + JavaScript simple                   | Equivalente a la capa que en MEAN/MERN ocupan Angular o React     |
| Base de datos | (a definir por cada equipo en el Sprint 1) | Relacional o NoSQL, según los criterios vistos hoy                |

No hay entregable de estudiantes: el objetivo es reconocer que ya han trabajado, sin nombrarlo, con piezas de un stack completo — y saber justificar la pieza que falta para el proyecto propio.

CIERRE DE UNIDAD 1

## Para la próxima semana

- Definir, en equipo, el stack tecnológico completo que usará el proyecto de Fábrica Escuela para el Sprint 1, justificando cada capa con los criterios de hoy.
- Repasar todo lo visto en la Unidad 1: frontend/backend/fullstack, definiciones de arquitectura, historia y evolución, diseño vs. arquitectura, UML, el rol del arquitecto, requisitos no funcionales y conectores.

**La Unidad 2, que empieza en la Semana 6, recorre el catálogo completo de patrones arquitectónicos POSA — la unidad más extensa del curso.**

